

Task 2: Client Lead Management System (Mini CRM)

Issued By: Future Interns | Project Blueprint & Implementation Manual

1. About the Task & Objectives

In modern digital ecosystems, managing client acquisition efficiently determines a business's growth velocity. Every time a prospective user completes a contact form, requesting an intake consultation or software quote, that data must be safely stored, categorized, monitored, and optimized down a sales pipeline.

This project focuses on architecting a multi-tier **Client Lead Management System (Mini CRM)**. This serves as the administrative core handling backend infrastructure, custom data routing pipelines, persistent storage rules, and a restricted-access workspace visualization module.

Operational Milestones:

- **Data Ingestion:** Capture asynchronous, raw user data coming from public-facing HTML forms.
- **State Orchestration:** Program structured lifecycle steps to transition leads cleanly (`new` → `contacted` → `converted`).
- **Audit Logging:** Maintain a flexible internal database tracking communications, milestones, and custom sales staff summaries.
- **Protected Dashboard:** Create an elegant frontend table view featuring data summaries, sorting operations, and instant database modifications.

2. Comprehensive System Architecture & Tech Stack

The system utilizes a decoupled, standard JavaScript engineering blueprint to ensure scalability, ease of local testing, and clean data routing:

- **Frontend View Layer:** Built utilizing `React.js`, leveraging state hooks and programmatic DOM renderings to represent database structures inside a matrix view layout.
- **Backend Application Layer:** Rooted in `Node.js` alongside `Express.js` to handle routing, request formatting validations, JSON processing middleware, and functional controller patterns.
- **Data Persistence Layer:** Structured via `MongoDB` using an explicit schema framework powered by `Mongoose ODM`.

 **Real-World Context:** This framework replicates enterprise architectural strategies used across SaaS startups, creative agencies, and production-scale B2B infrastructures to manage cold lead pipelines and optimize deal closing frequencies.

3. Database Layer (MongoDB Schemas)

Save the code below inside your local directory layout as `models/Lead.js`. This layout strictly implements validation structures to ensure database sanity across every incoming submission string.

```
const mongoose = require('mongoose');

const LeadSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Client baseline name is strictly required.'],
  },
  email: {
    type: String,
    required: [true, 'Contact parameter email is strictly required.'],
    match: [/^\S+@\S+\.\S+$/, 'Please use a valid email schema validation pattern.'],
  },
  source: {
    type: String,
    default: 'Web Core Contact Form'
  },
  status: {
    type: String,
    enum: ['new', 'contacted', 'converted'],
    default: 'new'
  },
  notes: {
    type: String,
    default: ''
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('Lead', LeadSchema);
```

4. Backend Server Implementation (Node.js & Express)

Save this structure as `server.js`. This acts as your microservices gateway, running data parsing middleware and binding to port configurations.

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const Lead = require('./models/Lead');
```

```

const app = express();
app.use(express.json());
app.use(cors());

// Establish Database Communications
mongoose.connect('mongodb://localhost:27017/mini_crm_db')
  .then(() => console.log('✅ Synchronized with MongoDB Production Cluster'))
  .catch(err => console.error('❌ Database connectivity breakdown:', err));

// Endpoint 1: Intake public contact data submissions
app.post('/api/leads', async (req, res) => {
  try {
    const { name, email, source } = req.body;
    const incomingLead = new Lead({ name, email, source });
    await incomingLead.save();
    res.status(201).json({ success: true, message: 'Lead safely written to disk.', data: incomingLead });
  } catch (err) {
    res.status(400).json({ success: false, error: err.message });
  }
});

// Endpoint 2: Administrative fetch querying full data arrays
app.get('/api/leads', async (req, res) => {
  try {
    const records = await Lead.find().sort({ createdAt: -1 });
    res.status(200).json({ success: true, data: records });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// Endpoint 3: Partial record update modifying status pipeline metadata or tracking notes
app.patch('/api/leads/:id', async (req, res) => {
  try {
    const { status, notes } = req.body;
    const modifiedLead = await Lead.findByIdAndUpdate(
      req.params.id,
      { status, notes },
      { new: true, runValidators: true }
    );
    res.status(200).json({ success: true, message: 'Patch cycle updated database state.', data: modifiedLead });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

const APP_PORT = 5000;
app.listen(APP_PORT, () => console.log(`🚀 REST Engine running successfully on port ${APP_PORT}`));

```



5. Frontend CRM Admin Dashboard (React.js)

Save this as `src/Dashboard.js`. This interfaces with the backend APIs via fetch commands and tracks temporary modifications using state arrays.

```
import React, { useState, useEffect } from 'react';

const Dashboard = () => {
  const [leads, setLeads] = useState([]);
  const [localNotes, setLocalNotes] = useState({});

  const downloadLeadManifest = async () => {
    try {
      const response = await fetch('http://localhost:5000/api/leads');
      const dataPayload = await response.json();
      if (dataPayload.success) setLeads(dataPayload.data);
    } catch (error) {
      console.error('REST synchronization error:', error);
    }
  };

  useEffect(() => {
    downloadLeadManifest();
  }, []);

  const dispatchUpdate = async (leadId, parameters) => {
    try {
      const targetUrl = `http://localhost:5000/api/leads/${leadId}`;
      const patchOptions = {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(parameters)
      };
      const networkResult = await fetch(targetUrl, patchOptions);
      if (networkResult.ok) downloadLeadManifest();
    } catch (err) {
      console.error('Mutation failure:', err);
    }
  };

  return (
```



Client Workspace Lead Analytics Tracker

```
{leads.map((item) => (
  )))}
```

Full Name	Email Address	Origination Source	Pipeline Status
{item.name}	{item.email}	{item.source}	{item.status.toUpperCase}

3. Launch and Configure React Frontend Workspace:

```
cd ../frontend
npx create-react-app .
# Implement the Dashboard runtime into src/App.js and start:
npm start
```

 **Final Submission Tip:** Before sharing your live project repository link on LinkedIn and tagging *Future Interns*, confirm that your database configuration string links correctly and your `README.md` matches these setup rules!